

OPERATING SYSTEMS, DV1697/DV1698

LAB 1: PROCESSES, THREADS, SYNCHRONIZATION, AND MEMORY SYSTEMS

Members:

Ebba Bengtsson

Signe Nilsson

Ha Thanh Chu

I – Part 1:

1. Task 7 - pthread_create(), how to handle return values

The program creates several threads using pthread_create. The variable in the struct called squaredId was added. Each thread takes its own id, calculates id * id, and stores the result in the struct. In main, an array of threads and an array of argument structs, start all the threads, and wait for them using pthread_join. When a thread is finished, it prints the squared id that it calculated and it is also printed in the end of the main function.

2. Task 9 - Bank account, correction

Following the task requirements, our implementation ensures that the shared variable is protected for the shortest possible time. The shared variable bankAccountBalance is only locked when its value is modified, and unlocked immediately afterwards. This minimizes the duration of the critical section while still preserving correctness. We use pthread_mutex_lock and pthread_mutex_unlock to protect the shared resource.

3. Task 11 - Dining Professors, deadlock-free implementation

The four conditions for resource deadlock to occur are:

1. Mutual exclusion condition – the resource is either currently assigned to one and only one process or is available.
2. Hold-and-wait condition. A process holding resources can request new resources.
3. No-preemption condition. If a resource is previously granted it can not be taken away from a process, only the process itself can release it.
4. Circular wait condition. There are waiting processes forming a circular list where each process is waiting for a resource already held by the next process in the list.

(Tanenbaum, A.S. & Bos, H. 2015, p.440)

The condition we removed to get a deadlock-free program is the circular wait condition. As soon as a professor can't pick up the right chopstick, he releases the left and starts over by thinking and then picking up the left chopstick etc. according to the implementation described in task 10. This means the threads won't form any circular waiting lists. As the professor releases the left resource chopstick it is free to use as the right chopstick by the professor to the left. This means eventually all professors will be guaranteed to eat and won't end up in deadlock.

4. Task 13 - Parallel matrix multiplication

I added a function that creates parallel threads, one thread for each row in the matrix. Each thread works on its own row. I use pthread_create to start the threads and

pthread_join to wait for all of them. This lets the program work on many rows at the same time, which makes it faster.

The difference between 13 and 14:

In task 13, only the multiplication is done in parallel. This means we use several threads to compute the rows at the same time, which is faster than doing everything sequentially. But the initialization of the matrices is still done by the main thread alone, so task 13 is not as fast as task 14.

In task 14, much more work is done in parallel: the initialization of A, the initialization of B, and the multiplication all run with multiple threads. Because of this, task 14 can do more work at the same time compared to task 13.

When the matrix is small, task 13 can sometimes be a bit faster because task 14 creates more threads, and that overhead takes time. But when the matrix becomes larger, task 14 becomes clearly faster because it uses parallelism better when the amount of work increases.

5. Task 14 - Parallelization of the initialization

We add an argument to pass input data to the functions that create the matrices. Additionally, we use a helper function to create threads and assign each thread to a row of the matrix, and then join them after completion.

How long was the execution time of your parallel program? Which speedup did you obtain compared to the sequential version ($Speedup = T_{sequential} / T_{parallel}$)?

For **SIZE = 1024**, parallelizing matrix initialization (Task 14) provides little benefit. Most of the time, Task 13 is faster because the initialization cost is small compared to matrix multiplication, and thread overhead offsets the gain.

Times	Task 13 speedup	Task 14 speedup
1	$3.498 / 1.525 \approx 2.295$	$3.498 / 1.605 \approx 2.179$
2	$3.498 / 1.581 \approx 2.212$	$3.498 / 1.572 \approx 2.226$
3	$3.498 / 1.431 \approx 2.445$	$3.498 / 1.418 \approx 2.467$
4	$3.498 / 1.487 \approx 2.352$	$3.498 / 1.572 \approx 2.226$
5	$3.498 / 1.459 \approx 2.397$	$3.498 / 1.632 \approx 2.143$
6	$3.498 / 1.423 \approx 2.459$	$3.498 / 1.606 \approx 2.178$
7	$3.498 / 1.455 \approx 2.404$	$3.498 / 1.551 \approx 2.255$

For **SIZE = 2024**, the cost of initialization grows, and Task 14 achieves higher speedup due to parallelization.

Times	Task 13 speedup	Task 14 speedup
1	$59.505 / 18.219 \approx 3.26$	$59.505 / 14.442 \approx 4.12$
2	$59.505 / 18.419 \approx 3.23$	$59.505 / 13.577 \approx 4.38$

This demonstrates that parallelizing initialization becomes more effective as problem size increases

II – Part 2:

1. Task 17 - Implementation of FIFO page replacement

The program implements the **First-In, First-Out (FIFO)** page replacement algorithm. It reads the number of physical frames, the page size, and a file containing memory addresses. Each address is converted to a page number, and the program simulates loading these pages into RAM using an array that represents the frames.

For each page reference, the program checks whether the page is already in memory: if it is, no page fault occurs; if it is not, a page fault is counted. When there is still free space in RAM, the new page is loaded into the next free frame. Once RAM is full, the program evicts pages in strict arrival order, always replacing the oldest loaded page. At the end, it prints the total number page faults.

2. Task 20 - Implementation of LRU page replacement

The program implements the **Least Recently Used (LRU)** page replacement algorithm. It maintains two arrays: *pages* to store pages in RAM, and *last* to track the most recent access time of each page.

For each memory reference, the program checks if the page is in RAM: if yes, it updates its last access time; if no, a page fault occurs. When RAM is full, the least recently used page is evicted and replaced with the new page. The program counts total references and page faults, and outputs these results at the end.

3. Task 23 - Implementation of Optimal page replacement (Bélády's algorithm)

The program implements the optimal page replacement algorithm. It maintains an array *frames* to hold the pages in RAM.

For each memory reference, the program checks if the page is in RAM with the function *isHit*. If the page is in RAM, the program continues to the next memory reference. If not, a page fault occurs. If a page fault occurs the program uses the function *freeFrameAvailable* to see if there is any “unused” frame. If so, the current page is stored in that frame and continues with the next memory reference. If there is no frame not used, the program runs the function *doOptimalReplacement*. This function loops through the remainder of the memory references and stores the index of next occurrence of the memory references in array *nextUse*. If there is a memory space in array *frames* that is never used again, that place in *frames* is replaced with the current page. If not, the highest index in *nextUse* is found and that memoryspace in *frames* is replaced by the currentPage. As we have the knowledge of all memory references that will be used, this is the optimal replacement as this is the memory reference that will be used farthest in the future and thus won't be needed in RAM for the longest time of each of the memory references.

References

[1] Tanenbaum, A.S. & Bos, H. (2015). *Modern Operating Systems*. 4th edition. Pearson Education Limited.